

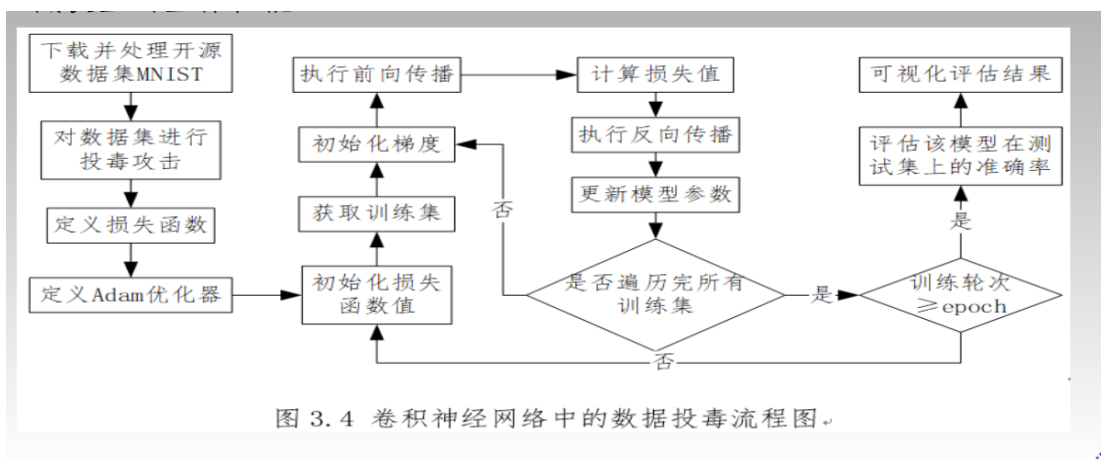
实验报告

实验题目：基于卷积神经网络的数据投毒攻击实验

课程题目：多媒体安全技术

1、实验目的和要求

深入理解数据投毒攻击及其对模型的影响，了解 AlexNet 模型结构及其运作原理，在 AlexNet 模型训练过程中对所使用的训练集进行投毒攻击，从而降低模型检测的准确率，并可视化训练过程和评估模型性能。



2、实验设备（环境）

pycharm2022.2.2、python3.10.9、torch2.7.0+cu128、torchvision0.22.0+cu128、numpy2.2.5、matplotlib3.10.1

3、实验步骤

● 定义 AlexNet 网络结构

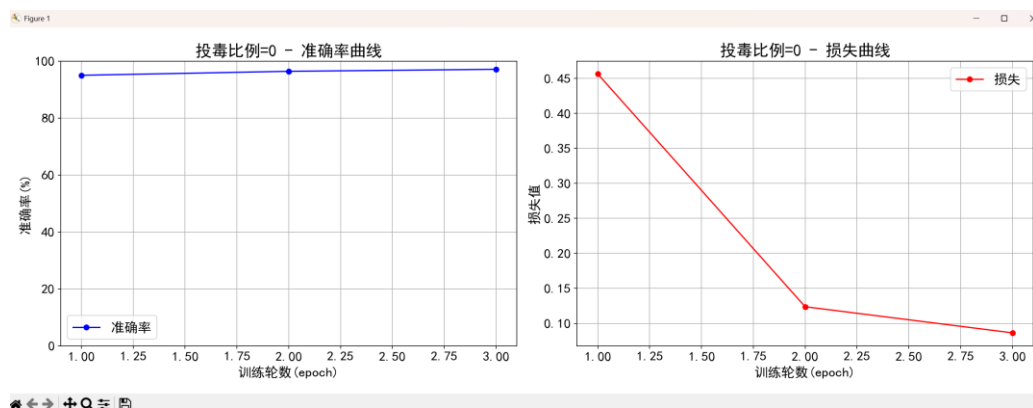
定义 AlexNet 网络结构，包含卷积层、最大池化层、ReLU 激活函数，以及三层全连接层（1024→512→10），输入通道为 1，适用于分类任务。

AlexNet 这个类特别针对处理 MNIST 数据集进行了调整，因为原始的 AlexNet 是为处理 227x227 像素的图像而设计的，而 MNIST 数据集中的图像大小是 28x28 像素。

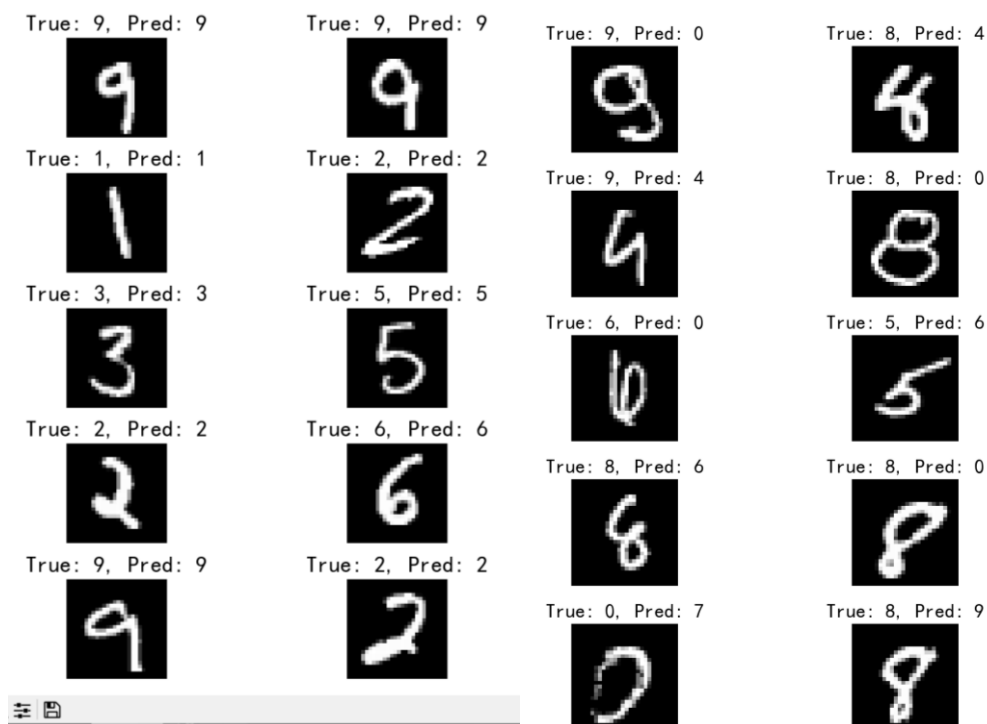
- 进行训练观察不同学习率得到的准确率有何不同，按照准确率的标准绘制损失函数线图横坐标为训练次数 epoch，纵坐标为损失值，并且展示分类正确的和错误的照片（调用 utils.py 中的 plot_correctly_classified_images 和 plot_misclassified_images 函数即可）。

```
100 # 绘制准确率和损失函数图
101 plt.rcParams['font.size'] = 16
102 plt.rcParams['font.sans-serif'] = ['SimHei']
103 plt.rcParams['axes.unicode_minus'] = False
104
105 # 创建一个包含两个子图的图表
106 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))
107
108 # 绘制准确率曲线
109 ax1.plot(range(1, len(clean_acc_list) + 1), clean_acc_list, label="准确率", marker='o',
110         ax1.set_title(f'投毒比例={args.poison_rate} - 准确率曲线')
111         ax1.set_xlabel('训练轮数(epoch)')
112         ax1.set_ylabel('准确率(%)')
113         ax1.set_ylim(0, 100)
114         ax1.legend()
115         ax1.grid(True)
116
117 # 绘制损失曲线
118 ax2.plot(range(1, len(loss_list) + 1), loss_list, label="损失", marker='o', linestyle='--')
119 ax2.set_title(f'投毒比例={args.poison_rate} - 损失曲线')
120 ax2.set_xlabel('训练轮数(epoch)')
121 ax2.set_ylabel('损失值')
122 ax2.legend()
123 ax2.grid(True)
124
125 plt.tight_layout() # 调整布局，防止子图重叠
126 plt.show()
```

- 投入受害者训练集污染受害者模型，修改投毒样本比例 0、50、100，绘制对应线图
■ 投毒比例为 0



随 epoch 增加，损失快速下降并趋于稳定，表明模型收敛，对于干净数据拟合良好。准确率接近 100%，说明模型对于干净样本的分类能力强，泛化性能优异。



模型准确识别数字，对清晰手写数字有高识别率。但对变形、模糊数字识别错误，反映出模型对极端样本的鲁棒性不足。

```
E:\作业\多媒体\1\Scripts\python.exe E:\作业\计算机设计\pythonProject\Poisoning_attack\main.py
开始对模型投毒.....
Epoch:1, loss: 0.4562
测试每一轮干净样本准确率: Epoch 1-----
干净样本准确率为: 95.00%

Epoch:2, loss: 0.1238
测试每一轮干净样本准确率: Epoch 2-----
干净样本准确率为: 96.40%

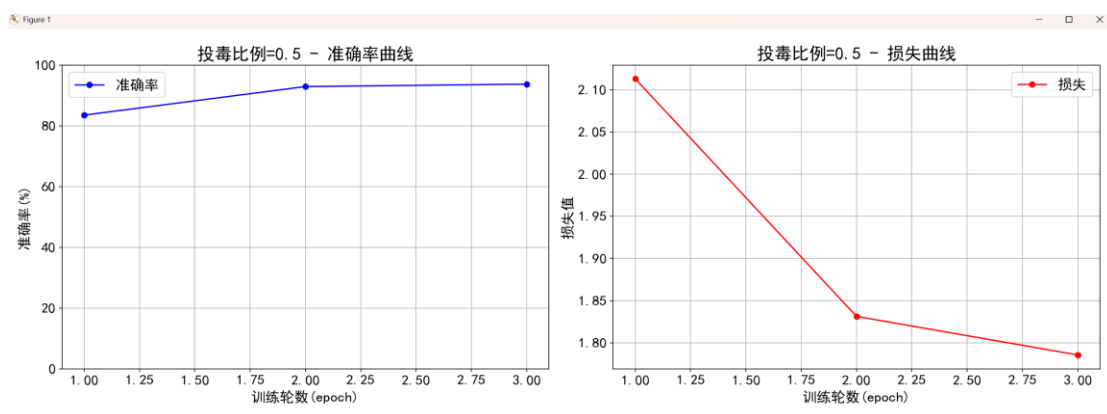
Epoch:3, loss: 0.0865
测试每一轮干净样本准确率: Epoch 3-----
干净样本准确率为: 97.10%

展示正确分类的图片...
展示错误分类的图片...

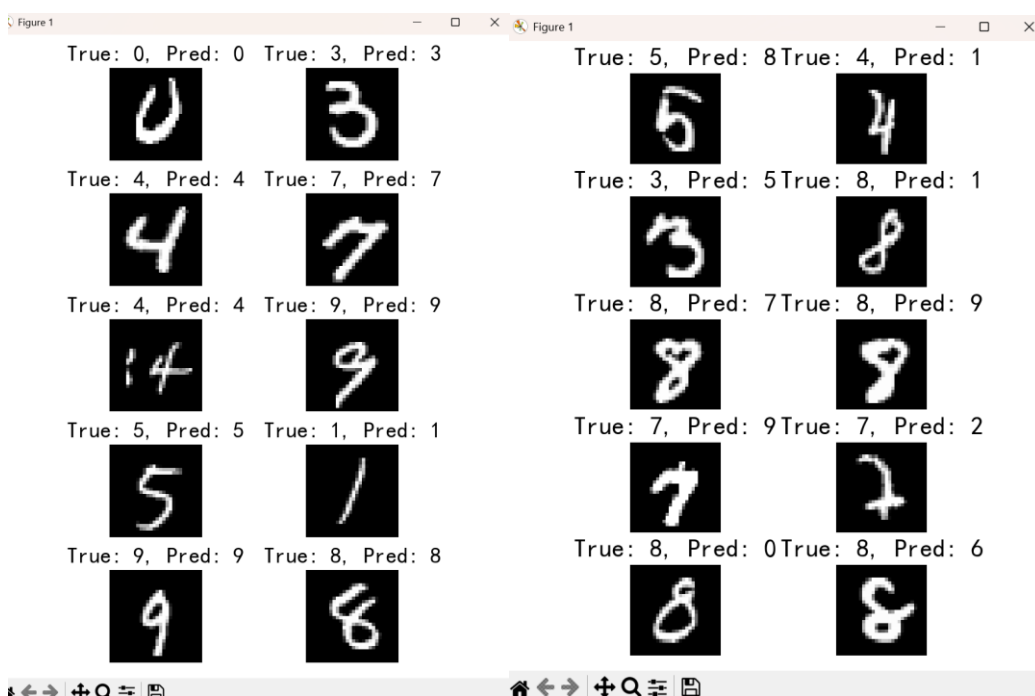
进程已结束,退出代码0
```

每轮 epoch 的干净样本准确率逐步提升 (95.00%→96.40%→97.10%), 验证了训练过程的稳定性。

■ 投毒比例为 50



准确率最终接近 100%，但初始更低，损失初始更高，表明投毒干扰训练但模型仍收敛。



保留对部分数字的识别能力，说明投毒未完全破坏模型，特定数字混淆，体现投毒对特征学习的篡改。

```
E:\作业\多媒体\1\Scripts\python.exe E:\作业\计算机设计\p
开始对模型投毒.....
Epoch:1, loss: 2.1130
测试每一轮干净样本准确率: Epoch 1-----
干净样本准确率为: 83.50%

Epoch:2, loss: 1.8313
测试每一轮干净样本准确率: Epoch 2-----
干净样本准确率为: 92.90%

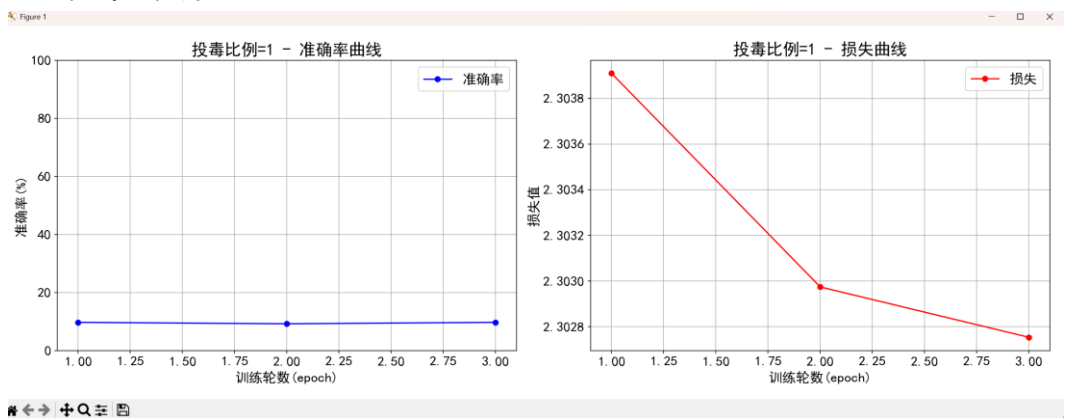
Epoch:3, loss: 1.7856
测试每一轮干净样本准确率: Epoch 3-----
干净样本准确率为: 93.70%

展示正确分类的图片...
展示错误分类的图片...

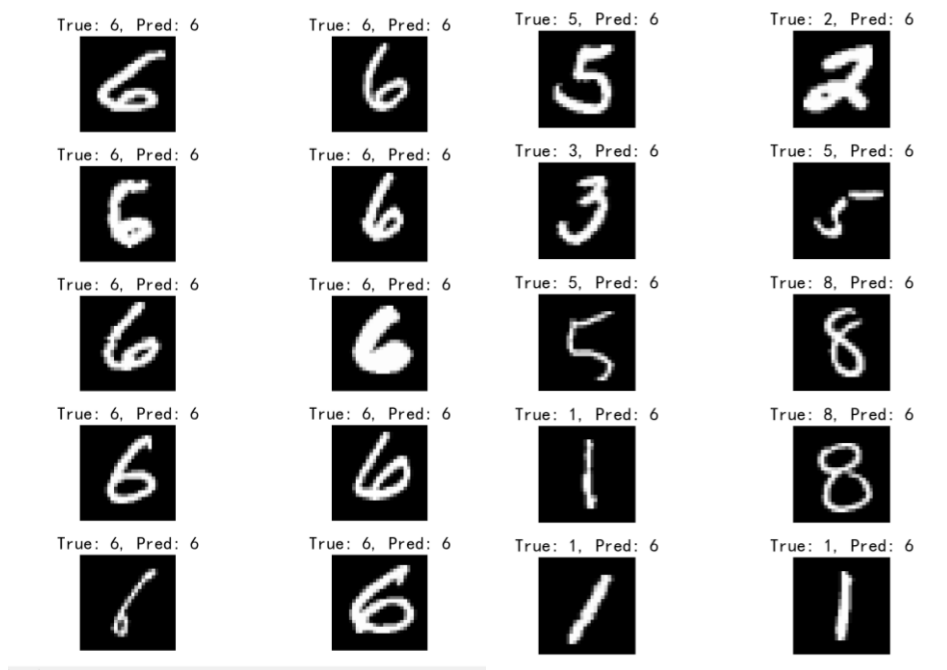
进程已结束,退出代码0
```

损失和准确率变化显示模型适应污染数据，干净样本准确率（93.7%）低于正常训练（97.1%），验证投毒降低泛化能力。投毒成功干扰模型，使干净样本分类性能下降，且攻击隐蔽，符合数据投毒攻击特征

■ 投毒比例为 100



准确率极低（约 10%），损失始终居高不下，说明模型完全被投毒数据误导，无法学习有效特征，反应训练数据完全污染，模型丧失分类能力。



仅对数字 6 有少量正确识别，表明投毒攻击将模型强制拟合到错误模式，其他数字（1、2、3、5、8 等）全部被误判为 6，体现投毒攻击的定向性。

```
E:\作业\多媒体\1\Scripts\python.exe E:\作业\计算机设计\pythonProject\Poisoning_attack\main.py
开始对模型投毒.....
Epoch:1, loss: 2.3039
测试每一轮干净样本准确率: Epoch 1-----
干净样本准确率为: 9.70%

Epoch:2, loss: 2.3030
测试每一轮干净样本准确率: Epoch 2-----
干净样本准确率为: 9.20%

Epoch:3, loss: 2.3028
测试每一轮干净样本准确率: Epoch 3-----
干净样本准确率为: 9.70%

展示正确分类的图片...
展示错误分类的图片...
```

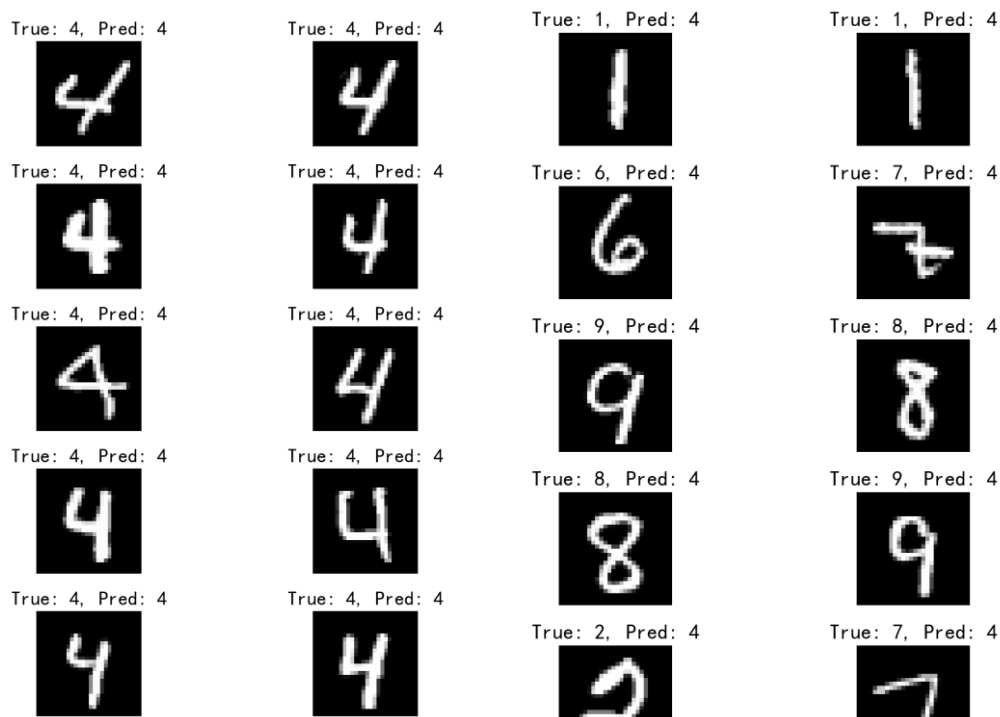
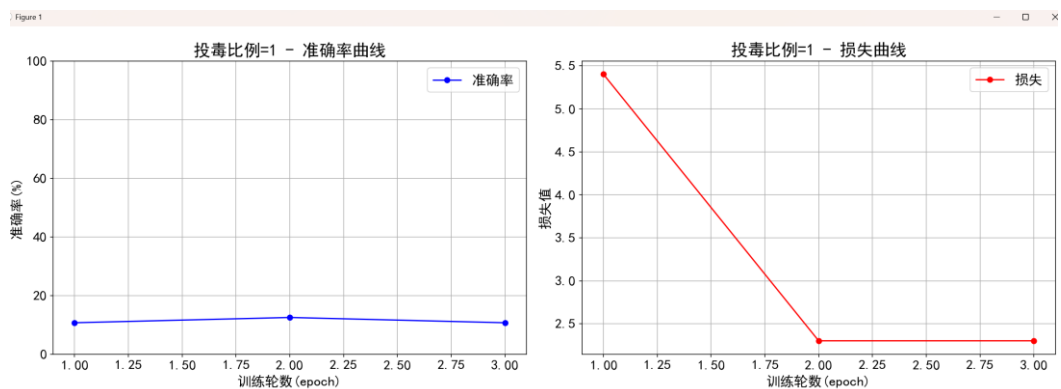
干净样本准确率仅约 9%, 验证模型完全失效, 投毒攻击成功破坏模型的泛化能力。

- 修改超参数，修改 epoch 寻找最优模型

高 lr: 训练速度快，但可能跳过最优解，对毒样本的拟合可能不充分。

低 lr: 收敛更稳定，但需更多 epochs，可能加剧对毒样本的过拟合。

- 修改学习率为 0.01



```

开始对模型投毒.....
Epoch:1, loss: 5.4025
测试每一轮干净样本准确率: Epoch 1-----
干净样本准确率为: 10.70%

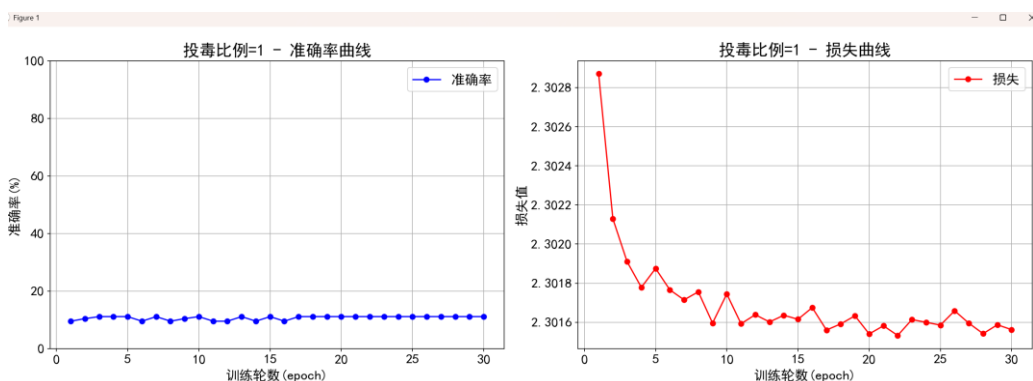
Epoch:2, loss: 2.3030
测试每一轮干净样本准确率: Epoch 2-----
干净样本准确率为: 12.50%

Epoch:3, loss: 2.3031
测试每一轮干净样本准确率: Epoch 3-----
干净样本准确率为: 10.70%
  
```

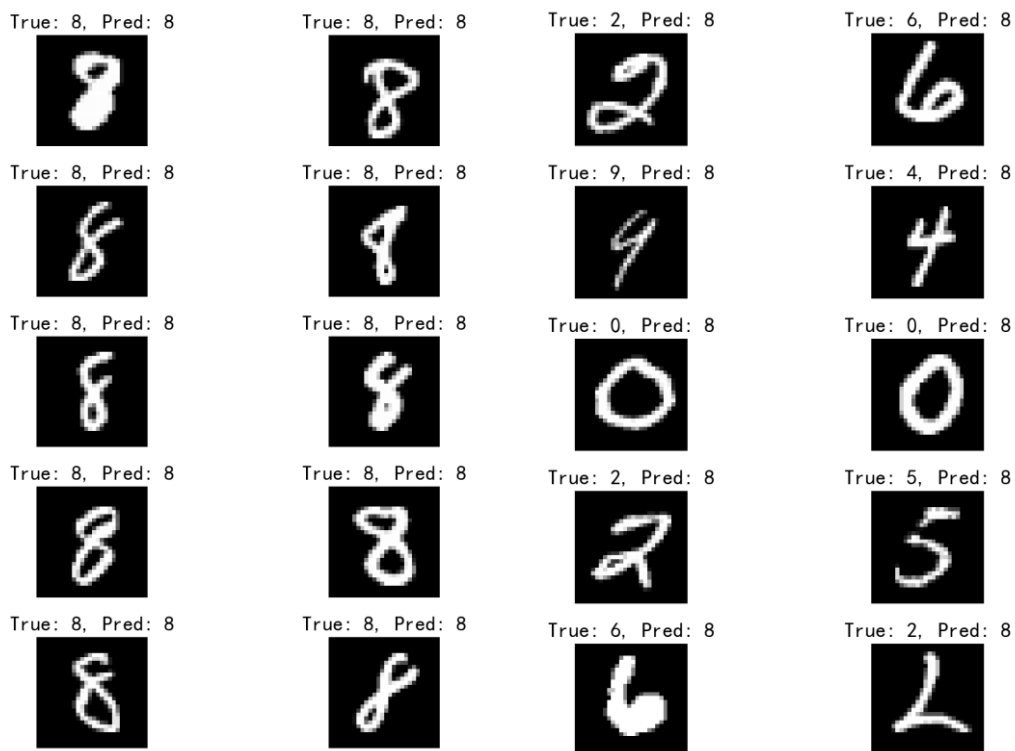
准确率微微提高了，投毒比例 1 时，模型完全失效，学习率调整无法修复数据污

染问题。

■ 再修改学习率为 0.001，迭代为 30 次



始终在 10% 左右波动，说明模型对干净样本几乎完全无法正确分类，投毒攻击彻底破坏了模型的特征学习能力。



Epoch:27, loss: 2.3035

测试每一轮干净样本准确率: Epoch 27-----

干净样本准确率为: 10.90%

Epoch:28, loss: 2.3032

测试每一轮干净样本准确率: Epoch 28-----

干净样本准确率为: 10.30%

Epoch:29, loss: 2.3033

测试每一轮干净样本准确率: Epoch 29-----

干净样本准确率为: 10.90%

Epoch:30, loss: 2.3036

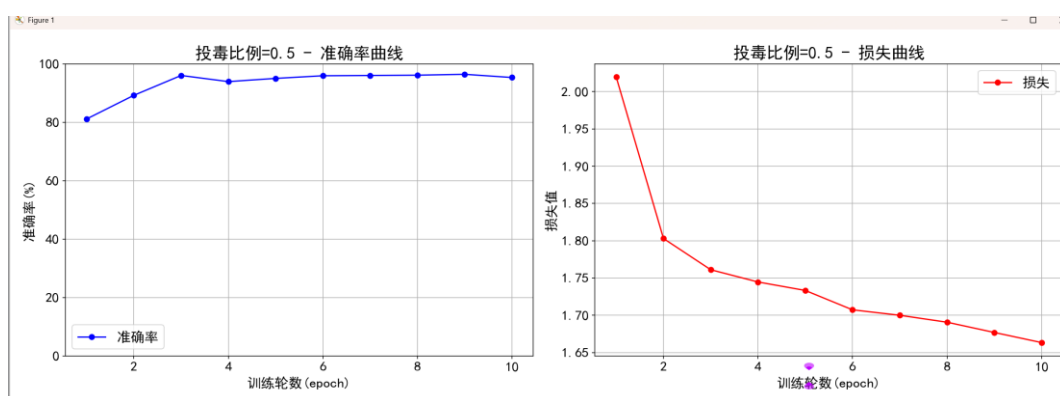
测试每一轮干净样本准确率: Epoch 30-----

干净样本准确率为: 10.90%

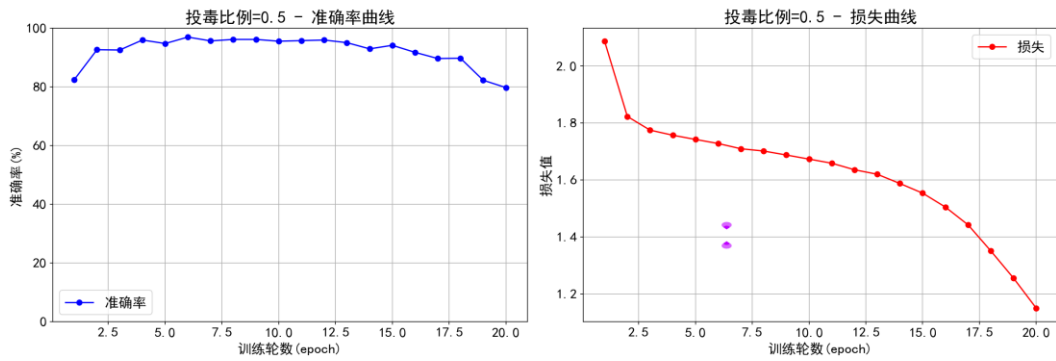
准确率其实没有什么变换, 较小的学习率使模型更新更谨慎, 避免训练初期的梯度爆炸, 确保损失稳定下降, 但最终仍无法恢复对干净样本的识别能力。足够的迭代次数让模型充分拟合投毒数据, 形成错误的分类模式, 进一步证明投毒攻击在全量污染数据下的破坏力。

4、实验结果

- 寻找最优 epoch,
通过以上实验过程我们可以知道投毒 100% 时模型失效, 无最优 epoch。
我们使用 50%投毒比例寻找最优参数
epoch 为 10:



epoch=3-4 时损失最低且准确率最高
epoch 为 20:



最后训练轮数 20 时准确率显著下降 (79.7%)，过拟合严重，模型对污染数据的错误特征强化，导致泛化能力大幅下降。

```
Epoch:17, loss: 1.4430
测试每一轮干净样本准确率: Epoch 17-----
干净样本准确率为: 89.60%

Epoch:18, loss: 1.3517
测试每一轮干净样本准确率: Epoch 18-----
干净样本准确率为: 89.70%

Epoch:19, loss: 1.2563
测试每一轮干净样本准确率: Epoch 19-----
干净样本准确率为: 82.20%

Epoch:20, loss: 1.1509
测试每一轮干净样本准确率: Epoch 20-----
干净样本准确率为: 79.70%
```

因此投毒 50%时，**epoch=3-4** 为最优，平衡了对干净数据的拟合与对污染数据的抗干扰能力

5、讨论和分析（分析实验原理、实验结果以及在实验中应注意的问题）

通过投毒比例 0、50、100 的对比实验，明确了：

部分污染 (50%)：可通过早停 (epoch=3-4) 优化模型，平衡拟合与抗污染，提升泛化能力。

全量污染 (100%)：模型失效，需从数据安全和模型鲁棒性层面防御。

在污染场景下，学习率和 epoch 的合理选择可缓解攻击影响，但无法根治 100%投毒的破坏，实验验证了数据投毒攻击的隐蔽性和破坏性。